NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

ADVANCED DATABASES

PROJECT REPORT

Project Title:

Graph Database–Enhanced Learning Assistant using Retrieval-Augmented Generation (RAG_Core)

Supervisor: **Tran Quang Toan**

Students: **Nguyen Son Hai** **11247282**
Hoang Thi Phuong Linh **11247308**

Class: **AI66B**
Group: **9**

Hanoi, May 2026

Abstract

This project builds a learning assistant that answers academic questions from user documents using a graph-database-oriented Retrieval-Augmented Generation (RAG) pipeline. Instead of storing extracted knowledge in relational tables, we model it as a property graph: entities become nodes and semantic relations become edges with weights, keywords, and provenance (chunk IDs and file paths). The system (RAG_Core) cleans and chunks documents, extracts entities/relations with an LLM (or a mock extractor for testing), merges duplicates, and supports keyword search, depth-limited neighborhood expansion, and multi-hop path finding. A Flask WebUI demonstrates end-to-end ingestion, graph construction, and hybrid question answering with an optional Vietnamese↔English translation layer (ALMA-13B via Ollama). Experiments show fast interactive retrieval on thousands of nodes and highlight how graph traversal reduces JOIN-heavy complexity compared to SQL. The design maps directly to Neo4j/Cypher for persistent storage and scale-out.

GitHub Repository:
https://github.com/PhuongLinhtla/retrieval-augmented-generation

Rubric-to-Section Mapping

Requirement (Rubric)	**Where to Find It**
Title page, members, supervisor	Section 1
Abstract (≈150 words)	Abstract (Section 1)
Project overview	Section 2.1
RDBMS (SQL) limitations analysis	Section 2.2
Rationale for chosen DB technology (Graph DB)	Section 2.3
Query-driven design (primary read/write ops)	Section 3.1.1
Graph data model (nodes/relationships/properties)	Section 3.1.2
System architecture diagram (Frontend→Backend→DB)	Section 3.2.1
DB-specific query/code snippets (graph traversal, indexing)	Section 3.2.2
Advanced features implementation	Section 3.3
Working application screenshots	Section 4.1
UI-to-Database mapping analysis	Section 4.2
Performance evaluation (response time benchmarks)	Section 4.3
Challenges faced	Section 5.1
Future enhancements	Section 5.2
Conclusion	Section 5.3

Contents

2 Introduction

2.1 Project Overview

Project title: Graph Database–Enhanced Learning Assistant using Retrieval-Augmented Generation (RAG_Core).
The project builds a document-grounded learning assistant that supports: (i) ingesting textbooks/notes (plain text, PDF, DOCX via conversion), (ii) transforming documents into structured knowledge, and (iii) answering questions using retrieval plus a Large Language Model (LLM). We implement a simplified LightRAG-style pipeline [1] with a strong emphasis on a *property graph* as the central database abstraction.
At a high level, the system workflow is:

1. **Chunking:** clean and split documents into overlapping chunks.
2. **Extraction:** extract entities and relationships per chunk (LLM-based; mock extractor available for testing).
3. **Graph construction:** merge entities into graph nodes and relations into graph edges with provenance.
4. **Retrieval:** perform keyword search and graph traversal (local/global/hybrid) to build a compact context.
5. **Generation:** ask the LLM to produce an answer grounded in the retrieved graph context.

2.2 RDBMS (SQL) Limitations in this Context

If MySQL/PostgreSQL were used to store and query the extracted knowledge, the system would face several bottlenecks:

- **JOIN-heavy multi-hop retrieval:** RAG questions often require neighborhood expansion and multi-hop reasoning ("A related to B related to C"). In SQL, this quickly becomes recursive CTEs and repeated self-joins, which are harder to optimize and maintain than native graph traversals.
- **Schema friction under evolving extraction:** entity types and attributes evolve as we add new document domains (e.g., adding *FORMULA*, *METHOD*, or *TECHNOLOGY* entities). In SQL, this often means ALTER TABLE cycles or sparse EAV patterns, increasing complexity and risking performance regressions.
- **Full-text + semantics integration:** practical RAG retrieval mixes keyword signals (exact terms) with structural evidence (graph connectivity) and optionally vector similarity. Implementing comparable functionality in pure SQL usually requires additional subsystems (external search engine + custom ranking) and more glue code.
- **Developer complexity:** representing provenance (chunk IDs, file paths) and edge-level metadata (keywords, weights) across many association tables leads to verbose queries and brittle application logic.

2.3 Rationale for Technology Choice

We choose a **graph database model (property graph)** as the primary data management paradigm because it matches the extracted knowledge structure natively:

- **Natural representation:** entities map to nodes; relationships map to edges; both carry flexible properties (type, description, keywords, weight, provenance).
- **Efficient traversal:** "local" retrieval is a bounded traversal around seed entities; "global" retrieval aggregates across broader neighborhoods. These are first-class operations in graph engines.
- **Schema flexibility:** adding new properties (e.g., confidence, timestamps, source counts) does not require schema migrations typical of relational systems.
- **Production path:** although RAG_Core stores the graph in-memory for the prototype, the same node/edge model maps directly to Neo4j labels and relationships, enabling persistent storage, indexing, and scale-out when needed.

3 Method

3.1 Data Modeling

3.1.1 Query-Driven Design Analysis

The database design is driven by the concrete read/write operations that the learning assistant must support.

Primary write operations (ingestion and indexing):

- **Ingest document:** convert input files to text (MarkItDown/PDF processor), then create ordered text chunks with overlap.
- **Extract entities:** for each chunk, extract entity candidates with attributes (type, description) and attach provenance (chunk ID, file path).
- **Extract relations:** for each chunk, extract relationships between entities with keywords, description, and an initial weight.
- **Merge into the graph:** upsert entities and relationships, merging duplicated names and accumulating edge weights across multiple mentions.

Primary read operations (retrieval for answering):

- **Keyword search:** find entities whose names/descriptions match query terms.
- **Local retrieval:** expand a bounded neighborhood (1–2 hops) from seed entities to gather precise facts.
- **Global retrieval:** retrieve broader sets of entities matching high-level themes.
- **Hybrid retrieval:** combine local and global candidate sets.
- **Path finding:** find multi-hop paths between two entities (for explainability and multi-hop reasoning).
- **Graph statistics:** compute density, degree distribution, and "most connected" entities to monitor graph health.

3.1.2 Graph Data Model (Neo4j-Style Property Graph)

We use a **property graph** model. This report presents the model in Neo4j terminology (labels, properties, relationships). The current prototype stores the graph in-memory (Python dictionaries), but the data model is directly portable to Neo4j.

3.2 Implementation & DB-Specific Queries

3.2.1 System Architecture

The system follows a simple Frontend→Backend→Database pipeline. The "database" is the property graph (in-memory for the prototype; Neo4j-compatible for persistence). LLM calls are served via Ollama (general model for extraction/answering; dedicated translation model for VN↔EN).

Element	Type	Properties (examples)
`:Entity`	Node label	`entity_id` (unique), `entity_type`, `description`, `created_at`, `source_ids[]` (chunk IDs), `file_paths[]`
`:Chunk`	Node label	`chunk_id` (unique), `content`, `tokens`, `chunk_order_index`, `file_path`
`:RELATED_TO`	Relationship	`description`, `keywords`, `weight`, `created_at`, `source_ids[]`, `file_paths[]`
`:MENTIONED_IN`	Relationship	Connects `:Entity`→`:Chunk` to represent provenance explicitly (optional; prototype stores provenance as arrays).

Table 1: Graph data model used by RAG_Core (Neo4j-compatible).

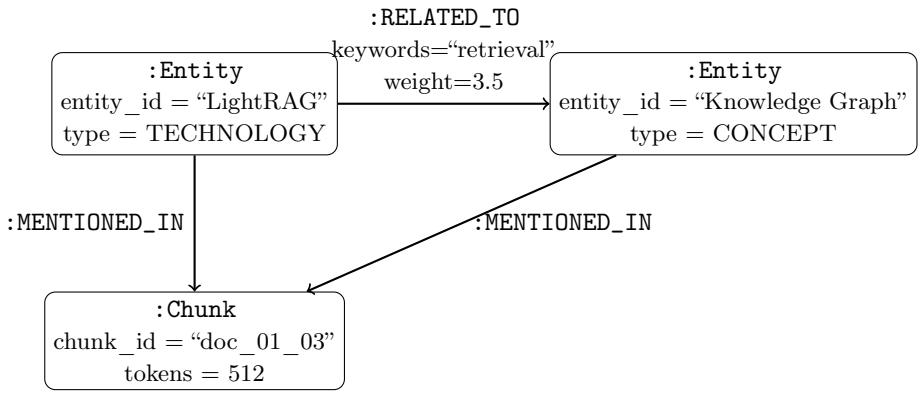

Figure 1: Illustration of the property graph: entities as nodes; relations as edges; chunks for provenance.

3.2.2 Core Code / Query Snippets

To highlight graph-database-oriented querying, we extract a few focused snippets (not the whole codebase).

(1) Graph upsert + merge (entity merge with provenance).

```python
# GraphBuilder._merge_entity (simplified)
if entity_name in self.nodes:
    node = self.nodes[entity_name]
    node.description = merge(node.description, [e.description for e in entities])
    node.source_ids = union(node.source_ids, [e.source_id for e in entities])
    node.file_paths = union(node.file_paths, [e.file_path for e in entities])
else:
    self.nodes[entity_name] = GraphNode(
        entity_id=entity_name,
        entity_type=entities[0].type,
        description=merge_descriptions([e.description for e in entities]),
        source_ids=[e.source_id for e in entities],
        file_paths=[e.file_path for e in entities],
    )
```

(2) Depth-limited neighborhood traversal (local retrieval).

```python
# QueryEngine.entity_search (BFS, depth-limited)
```

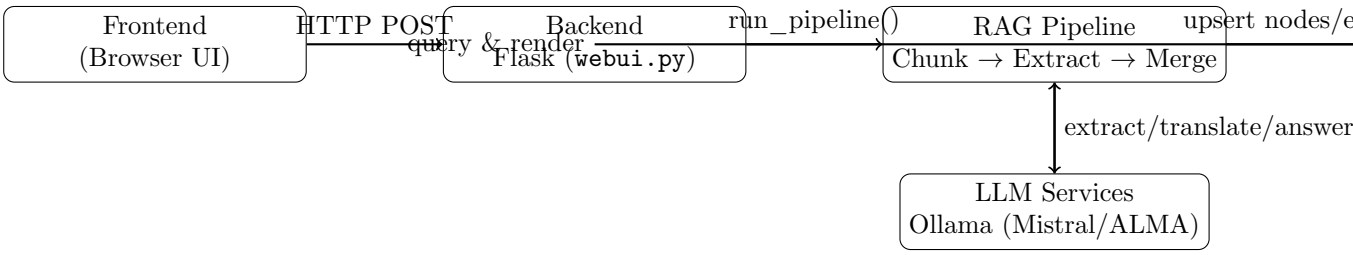

Figure 2: End-to-end architecture: UI to backend to graph store (Neo4j-compatible model).

```
to_visit = [(entity_id, 0)]
visited = set()
while to_visit:
    current, depth = to_visit.pop(0)
    if current in visited:
        continue
    visited.add(current)
    if depth < max_depth:
        for (u, v), edge in self.edges.items():
            if u == current and v not in visited:
                to_visit.append((v, depth + 1))
            elif v == current and u not in visited:
                to_visit.append((u, depth + 1))
```

(3) Neo4j/Cypher equivalents (production-ready backend).

```
// Full-text keyword search over Entity name + description
CALL db.index.fulltext.queryNodes('entityTextIndex', $q) YIELD node, score
RETURN node.entity_id, score ORDER BY score DESC LIMIT $k;

// 1-2 hop neighborhood expansion (undirected traversal)
MATCH (e:Entity {entity_id:$seed})-[r:RELATED_TO*1..2]-(n:Entity)
RETURN e, r, n;

// Multi-hop path finding (bounded)
MATCH p=(a:Entity {entity_id:$src})-[:RELATED_TO*1..3]-(b:Entity {entity_id:$tgt})
RETURN p LIMIT 5;
```

3.3 Advanced Features Implementation

Beyond basic graph storage and traversal, the system includes advanced features relevant to modern database applications:

- **Hybrid query modes:** `naive/local/global/hybrid` retrieval modes (configurable) to trade off precision vs. coverage.
- **Bilingual query support:** optional Vietnamese↔English translation using a dedicated LLM client (ALMA-13B) before/after retrieval.
- **Structured context assembly:** retrieval results are formatted as JSON blocks (entities, relations, chunks) to reduce prompt ambiguity and improve answer grounding.
- **Graph provenance tracking:** nodes/edges store `source_ids` and `file_paths` to support traceability from answers back to document chunks.

- **Accuracy benchmarking bridge:** an integration module compares RAG_Core outputs to LightRAG for validation in controlled tests.

4 Results

4.1 Core Feature Screenshots / GIFs

This section provides visual evidence of the working application (Flask WebUI for RAG_Core).

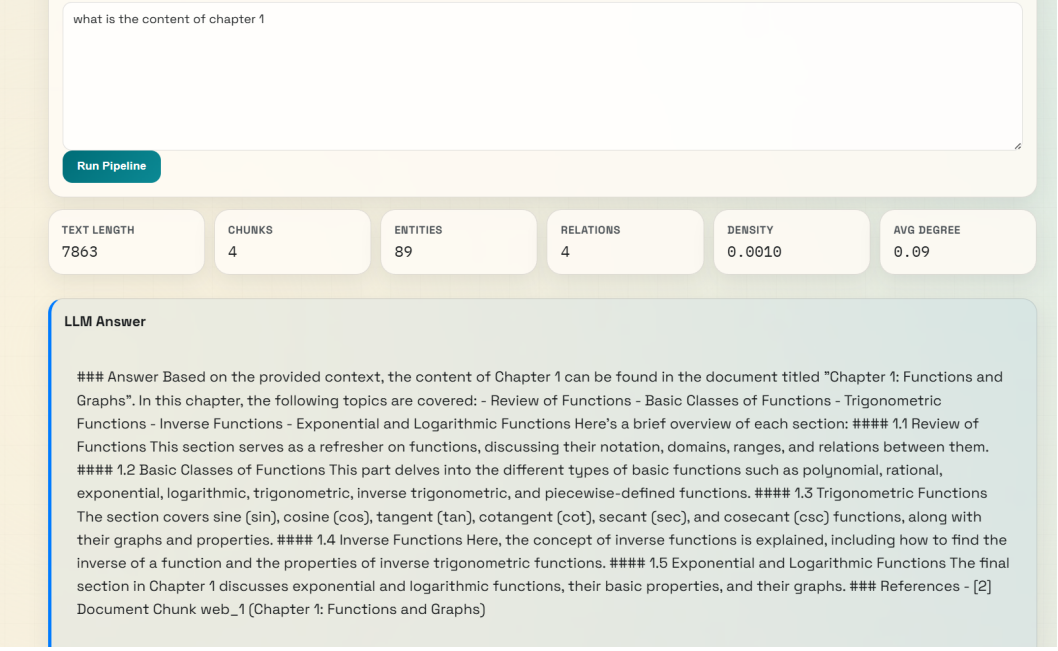

Figure 3: RAG_Core WebUI: running the pipeline and viewing a grounded LLM answer with graph statistics (chunks, entities, relations, density, average degree).

4.2 UI-to-Database Mapping Analysis

The WebUI actions map directly to property-graph operations:

- **Upload/Paste Document → Chunk Nodes:** when the user uploads a file or pastes text, the backend converts it to text, then applies overlapping chunking. Each chunk carries metadata (token count, order index, file path). In a persistent graph DB, this corresponds to creating `:Chunk` nodes.
- **Run Pipeline → Upsert Entities/Edges:** entity and relation extraction produces candidates per chunk. The backend upserts `:Entity` nodes and `:RELATED_TO` relationships, merging duplicates and accumulating weights across mentions.
- **Keyword Query → Graph Index/Search:** the *Keyword Query* field triggers an entity keyword search over node names/descriptions (prototype uses token overlap scoring; Neo4j can use a full-text index).
- **Entity Query → Neighborhood Expansion:** the *Entity Query* field triggers a depth-limited traversal (BFS) to retrieve the entity neighborhood (1–2 hops) for local reasoning.

- **Natural-Language Question → Hybrid Retrieval + LLM:** the *Natural Language Question* triggers translation (optional), hybrid retrieval (local + global candidates), structured context assembly (JSON blocks), and final answer generation by the LLM.

4.3 Performance Evaluation

We evaluate response time for core retrieval operations (micro-benchmarks). The goal is to validate that graph-oriented retrieval stays interactive as the number of entities and relations grows.

Benchmark setup (prototype):

- Chunking: fixed token window with overlap.
- Extraction: mock extractor (deterministic, no external LLM calls) to isolate database/retrieval cost.
- Storage: in-memory property graph (Python dicts) that mirrors a Neo4j-style model.
- Metric: wall-clock latency (ms), averaged over repeated runs.

Scale	Chunks	Nodes	Edges	Build graph (ms)	Keyword search (ms)	Neighborhood (ms)
Small	20	1000	20	3.21	0.52	0.01
Medium	100	5000	100	24.79	2.63	0.01
Large	400	20000	400	95.38	10.49	0.05

Table 2: Micro-benchmark results (average over 5 runs) using mock extraction and the in-memory property graph. Values vary by hardware and configuration.

The benchmarks demonstrate that neighborhood traversal and keyword search remain fast for interactive usage in the prototype. For production-scale datasets, we would persist the graph in Neo4j and rely on native indexing and optimized traversal execution.

5 Discussion

5.1 Challenges Faced

Developing a database-backed RAG assistant using a graph paradigm introduced several practical challenges:

- **Shifting from SQL mindset to traversal-first thinking:** the core retrieval logic is "start from a seed entity and traverse" rather than "JOIN tables". Designing queries as bounded traversals required careful control of depth, candidate explosion, and context size.
- **Entity identity and merge quality:** extracted entities are noisy (spelling variants, aliases, inconsistent casing). Merging nodes without over-collapsing distinct entities is non-trivial and impacts both precision and explainability.
- **Provenance representation trade-offs:** keeping provenance as arrays (chunk IDs/file paths) simplifies the prototype, but a production graph DB benefits from explicit `:Chunk` nodes and `:MENTIONED_IN` edges for richer auditability.
- **Benchmark reproducibility:** LLM-based extraction and translation introduce non-determinism and external latency. We used mock extraction for timing the database/-traversal layer, and separate tests for LLM-dependent stages.

5.2 Future Enhancements

Given more time, we would extend the system in the following directions:

- **Persist the graph in Neo4j:** enforce constraints on `:Entity(entity_id)`, build full-text indices, and move traversal to Cypher for larger datasets.
- **Add vector retrieval as a complementary index:** integrate FAISS/Milvus for dense similarity search and fuse it with graph traversal for stronger hybrid retrieval.
- **Incremental updates and caching:** support streaming ingestion, document deletion, and response caching for repeated queries.
- **Stronger ranking/explainability:** use learned rerankers and expose retrieved paths/-subgraphs to users as evidence.

5.3 Conclusion

This project demonstrates that modeling extracted knowledge as a property graph provides a natural and efficient foundation for RAG systems. Compared to a relational design, graph storage simplifies multi-hop retrieval and reduces query and code complexity. The RAG_Core prototype validates the end-to-end pipeline (ingestion, extraction, graph merge, traversal-based retrieval, and answer generation), and the same data model can be persisted to Neo4j for production-scale workloads.

References

[1] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation. 2024.